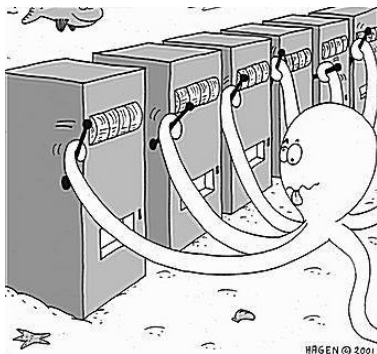


Multi-Armed Bandits

Jussi Keppo, Hong Ming Tan, and Linsheng Zhuang

National University of Singapore

2023



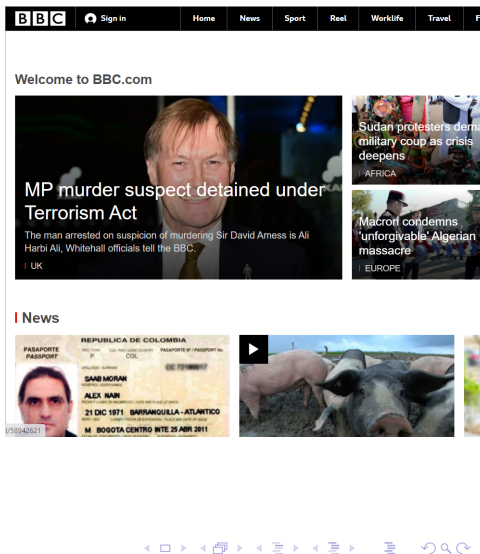
- A game: <https://cse442-17f.github.io/LinUCB/>
- Simple but very powerful framework for algorithms that make decisions over time under uncertainty

Examples

concrete albeit very stylized

News site When a new user arrives, a website picks an article to show, and observes whether the user clicks.

Pricing When a new customer arrives, the store chooses a price offered to this customer. The customer buys (or not) and leaves forever.



Examples

concrete albeit very stylized

Investment Each morning, you choose one stock to invest \$1 into. At the end of the day, you observe the change in value for each stock.

Canteen ?



The basic idea

- K possible actions/arms
- T rounds
- Each round, the algorithm chooses an arm and collects a reward for this arm
- Reward is drawn **independently** from an unknown distribution which is **fixed** (i.i.d rewards, depends only on the chosen arm)

Back to our examples

Example	Action	Reward
News site	an article to display	1 if clicked, 0 o.w.
Pricing	a price p to offer	p if sale, 0 o.w.
Investment	a stock to invest	change in value during the day
Canteen	?	?

- Observe reward for chosen arm
- Unobserved: reward for other arms that were not chosen. What should we do?

- Observe reward for chosen arm
- Unobserved: reward for other arms that were not chosen. What should we do?
- Algorithm typically needs to *explore*: try out different arms to acquire new information
- If you always eat from the same stall, how do you know if other stalls taste better?

Exploration-exploitation

- We have trade-off between exploration and *exploitation*: making optimal decisions based on the available information.
- This trade-off is essential in multi-armed bandits (MAB).
- The algorithm strives to learn which arms are best while not spending too much time exploring.

Auxiliary feedback

Sometimes, we get more information for free

- What feedback is available to the algorithm after each round, other than the reward for the chosen arm?

Example	Auxiliary feedback	Rewards for other arms?
News site	NA	no (bandit feedback)
Pricing	sale $\implies$ sale at any lower price, no sale $\implies$ no sale at any higher price	yes, for some arms, but not for all (partial feedback)
Investment	change in value of all other stocks	yes, for all arms (full feedback)
Canteen	?	?

Three types of feedback

Bandit feedback When the algorithm observes the reward of the chosen arm

Partial feedback When some information is revealed, in addition to the reward of the chosen arm

Full feedback When the algorithm observes the rewards of all the arms

We will focus on problems with bandit feedback.

Rewards model

Where do the rewards come from?

I.I.D. Reward for each arm is drawn independently from a fixed distribution that depends on the arm but not on the round

Adversarial Rewards can be arbitrary, as if they are chosen by an "adversary" that tries to fool the algorithm

Constrained adversary

Rewards are chosen by an adversary that is subject to some constraints, e.g., reward of each arm can change at most a few times

Stochastic (beyond I.I.D)

Rewards evolves over time as a random process, e.g., a random walk

Contextual bandits

- In each round, we may observe some *context* before choosing an action
 - ▶ often comprises known properties of the current user
 - ▶ allows for personalized actions

Example	Context
News site	user location and demographics
Pricing	store location, customer demographics
Investment	current state of the economy
Canteen	?

- Different objective: instead of learning the best arm, we learn the best *policy*

Application domains

- MAB problems arise in many domains
- Originally: design of "ethical" medical trials, to attain useful scientific data while minimizing harm to patients
- Modern applications concern the Web
 - ▶ tuning look and feel of website
 - ▶ optimizing web search results

LinkedIn Marketing Solutions shared.

Today's top-performing leaders are social leaders – here's how to join them:
<https://linkd.in/dnPCYE>



LinkedIn Marketing Solutions shared.

Today's top-performing leaders are social leaders – here's how to join them:
<https://linkd.in/dnH7B7>



A Multi-Armed Bandit for Recommendations at Netflix

Jaya Kawale
Elliot Chow

NETFLIX

- Recommender systems can use exploration to improve recommendations for movies, restaurants, hotels, etc.
- Procurement agency can optimize bids
- Teach a robot to perform tasks better

Application domain	Action	Reward
medical trials	which drug to prescribe	health outcome.
web design	<i>e.g.</i> , font color or page layout	#clicks.
content optimization	which items/articles to emphasize	#clicks.
web search	search results for a given query	1 if the user is satisfied.
advertisement	which ad to display	revenue from ads.
recommender systems	<i>e.g.</i> , which movie to watch	1 if follows recommendation.
sales optimization	which products to offer at which prices	revenue.
procurement	which items to buy at which prices	#items procured
auction/market design	<i>e.g.</i> , which reserve price to use	revenue
crowdsourcing	which tasks to give to which workers, and at which prices	1 if task completed at sufficient quality.
datacenter design	<i>e.g.</i> , which server to route the job to	job completion time.
Internet	<i>e.g.</i> , which TCP settings to use?	connection quality.
radio networks	which radio frequency to use?	1 if successful transmission.
robot control	a “strategy” for a given task	job completion time.

Stochastic Bandits and Notation

- I.I.D. rewards
- K possible actions/arms a
- $\mathcal{A}$ is the set of all arms
- T rounds, t for the t -th round
- For each a , there is a reward r_t^a following distribution $\mathcal{D}_a$ over reals. Distributions are initially unknown to the algorithm
- Mean reward of arm a is $\mu(a) := \mathbb{E}_{\mathcal{D}_a}(r_t^a)$
- Best mean reward is $\mu^* := \max_{a \in \mathcal{A}} \mu(a)$
- Gap of arm a , $\Delta(a) := \mu^* - \mu(a)$
- Optimal arm $a^* \Rightarrow \mu(a^*) = \mu^*$
- $[n]$ denotes $\{1, 2, \dots, n\}$

Problem protocol: Multi-armed bandits

Given: K arms, T rounds.

In each round $t \in [T]$:

1. Algorithm picks arm a_t .
2. Algorithm observes reward $r_t \in [0, 1]$ for the chosen arm.

- Goal is to maximize total reward over T rounds
- Three essential assumptions:
 - ▶ Only the reward for the selected action is observed (bandit feedback)
 - ▶ Reward for each action is I.I.D.
 - ▶ Per-round rewards are bounded; restricted to $[0, 1]$ for simplicity

How to know if an algorithm is doing a good job?

Regret

$$R(T) = \mu^* \cdot T - \sum_{t=1}^T \mu(a_t) \quad (1)$$

- This quantity is called regret at round T
- a_t is random as it may depend on randomness in rewards and/or in algorithm
- Hence $R(T)$ is also random
- We typically talk about expected regret $\mathbb{E}[R(T)]$
- Mainly care about the dependence of $\mathbb{E}[R(T)]$ on the time horizon T

What is the simplest algorithm you can think of?

Uniform exploration

- 1 Exploration phase: try each arm N times;
- 2 Select the arm $\hat{a}$ with the highest average reward (break ties arbitrarily);
- 3 Exploitation phase: play arm $\hat{a}$ in all remaining rounds.

Algorithm 1.1: Explore-First with parameter N .

- Fixed period of exploration
- What is bad about this algorithm?
- Any way to improve this algorithm?

ϵ -greedy

- With probability ϵ , choose an arm uniformly at random (explore)
- With probability $1-\epsilon$, choose the arm with the highest average reward so far (exploit)
- What is bad about this algorithm?
- Any way to improve this algorithm?

Adaptive ϵ -greedy

- With probability ϵ_t , choose an arm uniformly at random (explore)
- With probability $1 - \epsilon_t$, choose the arm with the highest average reward so far (exploit)
- For suitable choice of $\epsilon_t = \frac{1}{t}$, regret bounded by $O(\log(T))$, see Auer et al. (2002) and Cesa-Bianchi and Fischer (1998)
 - ▶ $1 + \log \log T + \log T + T^{0.5} + 5(\log T)^4 + 5T + 3T^2 + 2T^3 = O(T^3)$
- Vermorel and Mohri (2005) **did not** find any practical advantage to using adaptive ϵ methods
 - ▶ “One remarkable outcome of our experiments is that the most naive approach, the ϵ -greedy strategy, proves to be often hard to beat.”

Upper confidence bound algorithms - UCB1

- Play each arm once
- At every round t
 - ▶ Let $n_a(t)$ denote the number of times arm a has been chosen
 - ▶ Let $\hat{\mu}_a(t)$ be the estimated mean of arm a at round t
 - ▶ Denote the UCB index of arm a as

$$\hat{\mu}_a(t) + \sqrt{\frac{2 \ln(t)}{n_a(t)}}$$

- ▶ Choose the arm with the highest UCB index
 - ★ Let $X_{\hat{a}}$ be the observed reward of the chosen arm $\hat{a}$
 - ★ Update $\hat{\mu}_{\hat{a}}(t+1) = \frac{\hat{\mu}_{\hat{a}}(t) \times n_{\hat{a}}(t) + X_{\hat{a}}}{n_{\hat{a}}(t)+1}$
 - ★ Increment $n_{\hat{a}}(t+1) = n_{\hat{a}}(t) + 1$
- Regret is $O(\log T)$

References

- Auer P, Cesa-Bianchi N, Fischer P (2002) Finite-time analysis of the multiarmed bandit problem. *Machine learning* 47(2-3), 235–256.
- Cesa-Bianchi N, Fischer P (1998) Finite-time regret bounds for the multiarmed bandit problem. *ICML*, volume 98, 100–108 (Citeseer).
- Vermorel J, Mohri M (2005) Multi-armed bandit algorithms and empirical evaluation. *European conference on machine learning*, 437–448 (Springer).